

Zagdu Singh Charitable Trust's (Regd.)
Thakur Shyamnarayan Engineering College

Approved by All India Council for Technical Education (AICTE),
Approved By Government of Maharashtra
Affiliated to University of Mumbai



Second Year Engineering – AIML

Academic Year: 2026-2027
Full Stack Java Programming (FSJP)
(2113611)

Name	
Roll no.	
Program	

VISION

- *“Thakur Shyamnarayan Engineering College will strive to be a leading technical institute recognized for excellence in engineering education, research and innovation.”*

MISSION

- *To provide an excellent conducive environment that nurtures critical thinking, creativity and life-long learning amongst students.*
- *To promote innovative research ideas to address global challenges and contribute to technological advancement.*
- *To create entrepreneurs and competent technocrats to drive sustainable development and societal progress.*
- *To collaborate with other academic and research institutes and industries in order to strengthen multidisciplinary education and research.*



Certificate

This is to certify that Mr. / Ms.....
Roll No..... of **Third Semester** of
Engineering, **Thakur Shyamnarayan Engineering College**, Kandivali East,
Mumbai has completed the **Mini Project** satisfactorily in Course **Full Stack
Java Programming(FSJP) (2113611)** as per the prescribed syllabus of
University of Mumbai for the academic year 2026 to 2027

Place:

Date:

Course Faculty
Mrs. Poonam Joshi

HOD
Dr. Nirmala Kamble

Principal
Dr. S. M. Ganeshari

Seal of the Institute

Course Name: Full Stack Java Programming

Course Code: 2113611

Course Objectives

1.	Familiarize with Basic OOP concepts in Java,
2.	Understand the concepts of inheritance and exceptions in java,
3.	Design and implement programs involving Client and Server Side Programming,
4.	Describe and utilize the functioning of DOM and Java script,
5.	Study different design patterns in web programming and understand the working of react framework,
6.	To describe the Spring Framework and implement the related case studies.

Course Outcomes

CO 1	Understand and apply the fundamentals of Java Programming and Object-Oriented Programming,
CO 2	Analyze and Illustrate Inheritance and Exception Handling Mechanisms,
CO 3	Elaborate and design applications using Client and Server Side Programming,
CO 4	Understand the concepts in JavaScript for interactive Web Development,
CO 5	Implement the real-world application development in web programming using React,
CO 6	Design and Develop Enterprise-Level Applications Using the Spring Framework.

ABSTRACT

In contemporary digital communication, systematic organization, retrieval, and persistence of personal and corporate contact directories are crucial. Traditional contact management relying on volatile text files, flat spreadsheets, or physical phonebooks suffers from data redundancy, lack of structured validation, and inefficient querying as the number of records grows.

This mini-project presents the design and realization of a robust, modular, and optimized **Contact Management System (CMS)** developed using **Core Java (JDK 17+)**, **Java Database Connectivity (JDBC)**, and a **MySQL Relational Database Management System (RDBMS)**. The application separates the contact entity, persistence logic, database connection handling, and console interface through a small DAO-based structure.

Key implementation practices include parameterized SQL using **PreparedStatement**, automatic resource management through **try-with-resources**, input validation, and search by name or phone. The project intentionally avoids Spring, Hibernate, REST APIs, and other frameworks so that the core Java and JDBC concepts remain visible and easy to explain during a college viva.

Keywords: Core Java, JDBC, MySQL, OOP, CRUD, DAO, Contact Management, Console Application

TABLE OF CONTENTS

Chapter / Section	Title	Page
1	Introduction	1
1.1	Background	1
1.2	Technologies Used	1
2	Literature Survey	2
2.1	Existing System	2
2.2	Analysis of Literature	2
3	Research Gap and Problem Statement	3
3.1	Research Gap	3
3.2	Aim and Objectives	3
3.3	Problem Statement	3
4	Design Methodology	4
4.1	Proposed System Framework	4
4.2	Database Design and Schema	5
4.3	Hardware and Software Requirements	6
4.4	System Design & Modeling	6
4.4.1	UML Diagrams	7
4.4.2	DFD	9
5	Implementation Plan & Results	10
5.1	Timeline Chart	10
5.2	Future Scope	11
5.3	Implementation Outputs & Test Results	12
	Conclusion	15
	References	16

LIST OF FIGURES

Figure No.	Figure Title	Page
1.1	Architecture of JDBC Connection and Layered System	1
4.1	Multi-tier Architecture (Presentation-DAO-Database)	4
4.2	Entity Relationship Diagram of Contact Management	5
4.3	UML Class Diagram	7
4.4	UML Use Case Diagram	7
4.5	UML Sequence Diagram for Contact Insertion	8
4.6	DFD Level 0 - Context Level	9
4.7	DFD Level 1 - Functional Decomposition	9
5.1	Main Application Menu Interface	12
5.2	Add New Contact Console Execution	12
5.3	View All Contacts Tabular Display	12
5.4	Search Contact by Keyword Execution	13
5.5	Update Contact Attribute Flow	13
5.6	Delete Contact with Confirmation Prompt	13
5.7	MySQL Database Record Verification	13

LIST OF TABLES

Table No.	Table Title	Page
TABLE I	Comparative Analysis of Existing vs. Proposed System	2
TABLE II	Software & Hardware Specification Requirements	6
TABLE III	Database Schema Definition for contacts Table	5
TABLE IV	Project Work Timeline	10
TABLE V	Test Cases and Verification Results	14

CHAPTER 1: INTRODUCTION

1.1 Background

Contact management is a fundamental requirement across personal productivity workflows and enterprise environments. A Contact Management System (CMS) enables users to store names, phone numbers, email IDs, physical addresses, and relational categories such as Friend, Work, Family, or Client within a centralized, queryable repository.

Historically, contact data was maintained through physical address books, text files such as `.txt`, or spreadsheet documents such as `.csv` and `.xlsx`. As the volume of contacts grows and Create, Read, Search, Update, and Delete operations become frequent, these approaches become harder to manage and validate.

This project implements an academic-grade Contact Management System using **Core Java** and **MySQL** through the **JDBC** standard. The design keeps database operations separate from the console interface and stores all contact records persistently.

1.2 Technologies Used

Technology	Role in the Project
Core Java (JDK 17+)	OOP, classes, methods, collections, input handling, validation and exception handling.
JDBC	Java API used to establish connections and execute SQL statements.
MySQL 8.x	Relational database used for persistent contact storage.
MySQL Connector/J	JDBC driver connecting the Java application to MySQL.
XAMPP / phpMyAdmin	Optional local environment for running and inspecting the MySQL database.

Figure 1.1 Architecture of JDBC Connection and Layered System

Figure 4.1 Multi-tier Architecture



CHAPTER 2: LITERATURE SURVEY

2.1 Existing System

In typical small-scale and academic implementations, contact storage systems use one of three approaches:

1. In-Memory Arrays / Collections: Data is held in structures such as `ArrayList<Contact>`. The approach is simple, but all modifications are lost when the application terminates.

2. Flat File Storage: Contacts are written to text or serialized files. Searching generally requires scanning records, while record-level updates and safe concurrent modifications require additional custom logic.

3. Spreadsheet Files: Excel or CSV files provide persistence and manual filtering, but they do not provide the same programmatic schema constraints, parameterized queries, or structured CRUD workflow as a relational database.

2.2 Analysis of Literature

Parameter	File / In-Memory	Spreadsheet	Proposed JDBC + MySQL
Persistence	Volatile or file based	Persistent local file	Persistent database
Search	Usually linear scan	Filtering / manual	SQL with LIKE
Security	Minimal	File-level controls	PreparedStatement
Modularity	Often monolithic	External to application	DAO-based separation
Data Integrity	Manual checks	Moderate	PK / NOT NULL constraints

The proposed solution is intentionally smaller than a commercial CRM while providing the database persistence and structured CRUD workflow required for a meaningful Java and JDBC academic project.

CHAPTER 3: RESEARCH GAP AND PROBLEM STATEMENT

3.1 Research Gap

Modern enterprise CRM platforms such as Salesforce and HubSpot provide extensive features, but they are much larger than what is necessary for a basic local contact directory. They also introduce cloud services, authentication systems, subscription models, and multiple software layers that can obscure the fundamental concepts being evaluated in a Core Java and JDBC course.

At the other extreme, introductory student programs often place SQL strings, input handling, validation, and database connection code in one large class. Such a design works for a demonstration but becomes difficult to read and maintain. The project therefore targets a middle ground: a compact implementation with clear separation of responsibilities but no unnecessary enterprise framework.

The implementation combines:

- Minimal footprint with no Spring or Hibernate dependency.
- A small DAO-based separation between application logic and database operations.
- PreparedStatement for parameterized SQL.
- try-with-resources for reliable JDBC resource management.
- Simple validation for names, phone numbers, and email addresses.

3.2 Aim and Objectives

Aim: To design, implement, and validate a secure and efficient console-based Contact Management System in Core Java using MySQL persistence over JDBC.

Specific Objectives:

- Implement an encapsulated Contact model containing identity and contact details.
- Centralize JDBC connection configuration in DatabaseConnection.
- Create ContactDAO to encapsulate CRUD SQL transactions.
- Validate strings, telephone numbers, and email patterns before database operations.
- Use PreparedStatement for all user-driven SQL operations.
- Ensure JDBC resources are closed using try-with-resources.

3.3 Problem Statement

“To develop an interactive, robust, and platform-independent Java application that interfaces with a MySQL relational database to provide full lifecycle management (Create, Read, Search, Update, and Delete) of contact directories with strict input validation, formatted tabular presentation, and secure transaction handling.”

CHAPTER 4: DESIGN METHODOLOGY

4.1 Proposed System Framework

The system adopts a lightweight three-tier architecture. The separation is conceptual rather than framework-driven, which keeps the implementation small.

- **Presentation Layer (Main.java):** Handles the console menu, user input, validation, and display formatting.
- **Data Access Layer (ContactDAO.java):** Encapsulates SQL statements and database CRUD operations.
- **Persistence Layer (MySQL):** Stores contact records in the contact_management database.

Figure 4.1 Multi-tier Architecture



The database connection utility is used by the DAO layer to obtain JDBC connections. The Contact model carries contact data between the interface and the DAO.

4.2 Database Design and Schema

The database uses a single contacts table because the current scope does not require relationships with other entities. The primary key is automatically generated so that users do not have to manage identifiers manually.

Column Name	Data Type	Constraints	Description
id	INT	PRIMARY KEY, AUTO_INCREMENT	Unique contact identifier
name	VARCHAR(100)	NOT NULL	Full name of the contact
phone	VARCHAR(20)	NOT NULL	Primary phone number
email	VARCHAR(100)	NULL	Optional email address
address	VARCHAR(255)	NULL	Residential or office address
category	VARCHAR(50)	NULL	Friend, Work, Family, etc.

Figure 4.2 Entity Relationship Diagram



Figure 4.2 Entity Relationship Diagram

DDL SQL Script

```
CREATE DATABASE IF NOT EXISTS contact_management;
USE contact_management;
```

```
CREATE TABLE IF NOT EXISTS contacts (
    id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(100) NOT NULL,
    phone VARCHAR(20) NOT NULL,
    email VARCHAR(100),
    address VARCHAR(255),
    category VARCHAR(50)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

4.3 Hardware and Software Requirements

Component	Specification / Version
Operating System	Windows 10 / 11, Linux (Ubuntu 20.04+), or macOS
Processor	Intel Core i3 / AMD Ryzen 3 or higher (x86_64)
RAM	Minimum 4 GB; 8 GB recommended
Hard Disk Space	Approximately 500 MB free space
Java Environment	OpenJDK / Oracle JDK 17+ recommended
Database Server	MySQL 8.x or compatible MySQL installation
JDBC Driver	MySQL Connector/J 8.x
IDE / Editor	VS Code / IntelliJ IDEA / Eclipse / Terminal

The project does not require a web server, cloud account, REST API, or frontend framework. This is intentional: the project is meant to demonstrate Java and JDBC fundamentals with the smallest practical technology stack.

4.4 System Design & Modeling

The implementation contains four primary Java classes:

- **Contact**: Entity POJO containing id, name, phone, email, address, and category.
- **DatabaseConnection**: Utility class that provides JDBC connections.
- **ContactDAO**: Data Access Object containing add, view, search, update, and delete operations.
- **Main**: Interactive console controller containing menu handling and input validation.

4.4.1 UML DIAGRAMS

Figure 4.3 UML Class Diagram

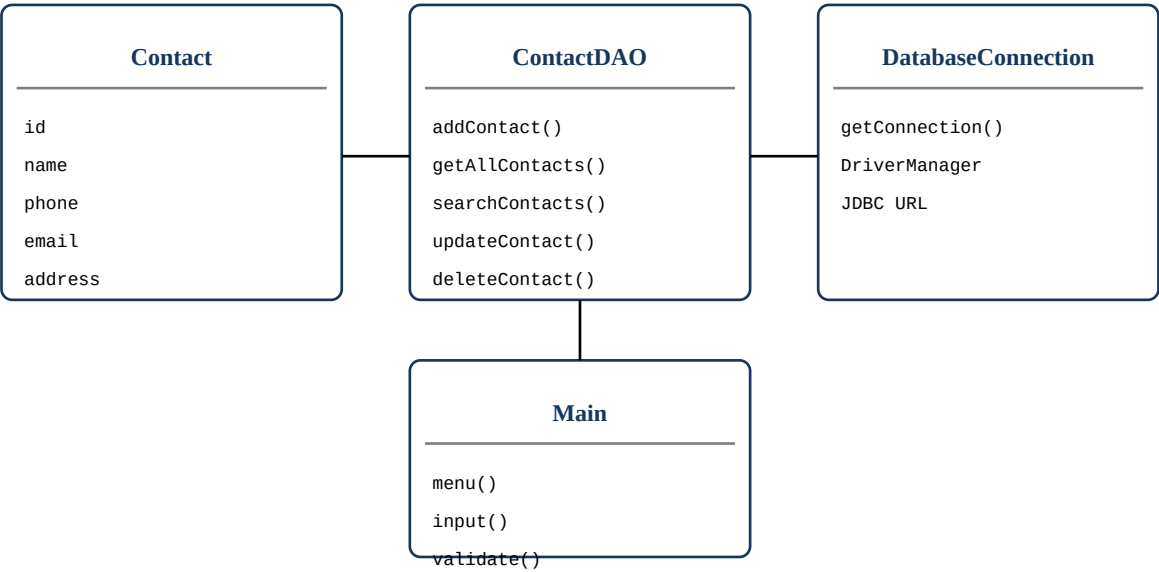


Figure 4.3 UML Class Diagram

Figure 4.4 UML Use Case Diagram

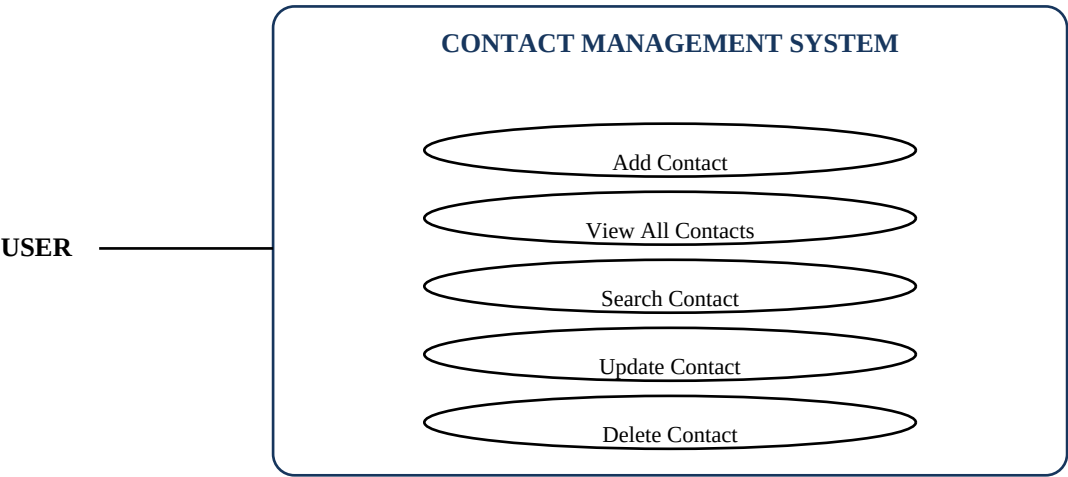


Figure 4.4 UML Use Case Diagram

Figure 4.5 UML Sequence Diagram for Contact Insertion

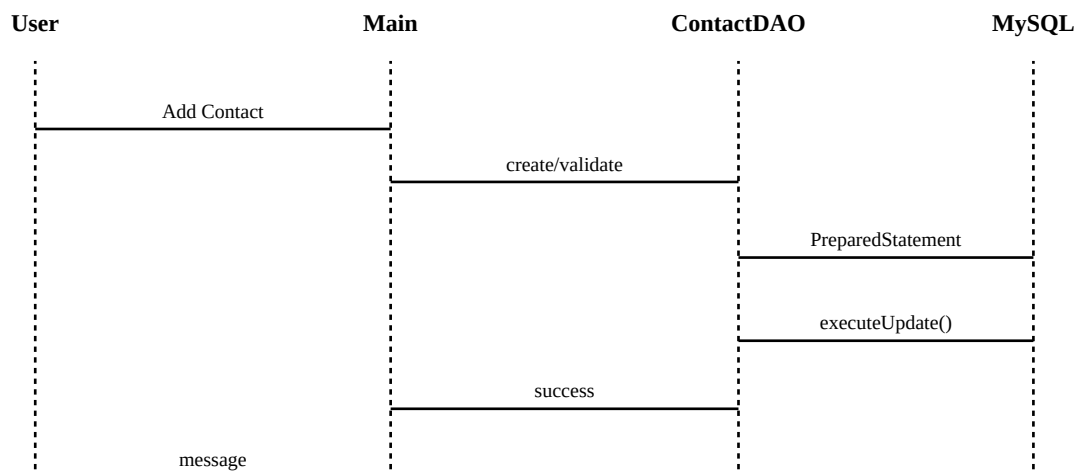


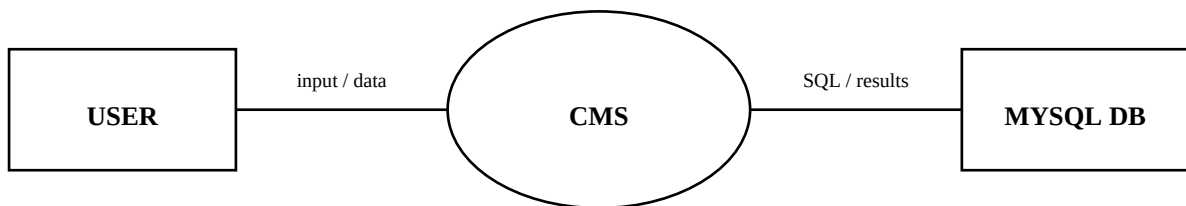
Figure 4.5 UML Sequence Diagram for Contact Insertion

4.4.2 DATA FLOW DIAGRAMS

DFD Level 0 (Context Level)

The context-level diagram represents the complete application as one process. The user sends operations and contact data to the system, while the system sends formatted results and alerts back to the user. SQL queries and database results form the communication between the application and MySQL.

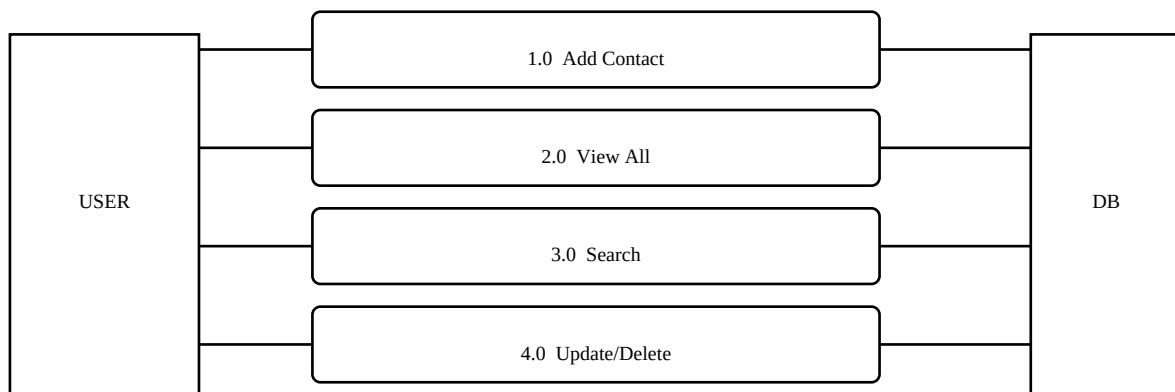
Figure 4.6 DFD Level 0 - Context Level



DFD Level 1 (Functional Decomposition)

The Level 1 diagram decomposes the application into four major functional processes: add, view, search, and update/delete.

Figure 4.7 DFD Level 1 - Functional Decomposition



CHAPTER 5: IMPLEMENTATION PLAN & RESULTS

5.1 Timeline Chart (Term-I and Term-II)

Phase	Activity / Milestone	Duration	Status
Phase 1	Requirement Analysis & Literature Survey	Weeks 1–2	Completed
Phase 2	Database Schema Formulation & DDL Scripting	Weeks 3–4	Completed
Phase 3	Core Java Architecture & DAO Implementation	Weeks 5–7	Completed
Phase 4	Console Interface, Validation & Error Handling	Weeks 8–9	Completed
Phase 5	Integration Testing & SQL Injection Validation	Weeks 10–11	Completed
Phase 6	Documentation, Code Refactoring & Report Preparation	Weeks 12–14	Completed

The timeline focuses on the actual learning objectives of the project: understanding the database schema, implementing JDBC operations, connecting the console workflow to the DAO layer, and testing the complete CRUD cycle.

5.2 IMPLEMENTATION PLAN FOR NEXT SEMESTER / FUTURE SCOPE

The current project intentionally remains a console-based Core Java + JDBC system. If the project is expanded in a later semester, the same core database concepts can be used as the foundation for a full-stack version.

- **Full-Stack Web Interface:** Replace the CLI with a responsive web interface using React.js and Tailwind CSS.
- **REST API Backend:** Expose the existing CRUD functionality through Spring Boot REST controllers.
- **ORM Layer:** Replace direct JDBC queries with Spring Data JPA / Hibernate if required by a later course.
- **Security and Authentication:** Add JWT-based multi-user authorization with Spring Security.
- **Import / Export:** Add CSV and VCF import/export for easier migration from existing contact lists.

These features are future scope only. Adding them to the current mini-project would make the implementation larger than necessary and would reduce the focus on Core Java and JDBC.

Implementation Output and Results

The following execution traces represent the expected application output for the implemented workflow. They are included as report figures in console format rather than being presented as fabricated screenshots.

5.3 IMPLEMENTATION OUTPUTS & TEST RESULTS

Figure 5.1 Main Application Menu Interface

```
=====
CONTACT MANAGEMENT SYSTEM
=====
1. Add Contact
2. View All Contacts
3. Search Contact
4. Update Contact
5. Delete Contact
6. Exit
Enter your choice:
```

Figure 5.2 Add New Contact Console Execution

```
Enter your choice: 1
Enter Name: Rahul Sharma
Enter Phone: 9876543210
Enter Email: rahul.sharma@example.com
Enter Address: Mumbai, Maharashtra
Enter Category: Friend

Contact added successfully.
```

Figure 5.3 View All Contacts Tabular Display

```
--- All Contacts ---
-----
ID | NAME           | PHONE       | EMAIL                           | CATEGORY
-----
1  | Rahul Sharma   | 9876543210 | rahul.sharma@example.com       | Friend
2  | Aman Patel     | 9876501234 | aman.patel@example.com         | Work
3  | Priya Singh    | 9123456780 | priya.singh@example.com        | Family
-----
Total Records: 3
```

Figure 5.4 Search Contact by Keyword Execution

--- Search Contact ---

Enter name or phone to search: Aman

```
-----
ID | NAME          | PHONE      | EMAIL                      | CATEGORY
-----
2  | Aman Patel    | 9876501234 | aman.patel@example.com    | Work
-----
Total Records: 1
```

Figure 5.5 Update Contact Attribute Flow

Enter Contact ID: 1

Enter new phone: 9988776655

Contact updated successfully.

Figure 5.6 Delete Contact with Confirmation Prompt

Enter Contact ID: 2

Are you sure you want to delete this contact? (Y/N): Y

Contact deleted successfully.

Figure 5.7 MySQL Database Record Verification

mysql> SELECT id, name, phone, email, category FROM contacts;

```
+----+-----+-----+-----+-----+
| id | name      | phone    | email                      | category |
+----+-----+-----+-----+-----+
| 1  | Rahul Sharma | 9988776655 | rahul.sharma@example.com | Friend   |
| 3  | Priya Singh  | 9123456780 | priya.singh@example.com   | Family   |
+----+-----+-----+-----+-----+
```

TEST RESULTS

ID	Feature Tested	Input Given	Expected Result	Actual Result	Status
TC-01	Add Contact	Valid fields	Record inserted with auto ID	Inserted successfully	PASS
TC-02	Empty Name	Name: ""	Error and re-request	Name cannot be empty	PASS
TC-03	Phone Validation	Phone: abc12	Reject invalid phone	Please enter valid phone	PASS
TC-04	Email Validation	notanemail	Reject invalid email	Validation message shown	PASS
TC-05	Search Keyword	98765	Match partial phone	Matching contacts displayed	PASS
TC-06	Update Field	ID 1, new phone	Phone updated	Record updated	PASS
TC-07	Delete Contact	ID 2, Y	Record removed	Deleted successfully	PASS
TC-08	Delete Abort	ID 1, N	Operation cancelled	Deletion cancelled	PASS

The test cases cover the essential CRUD lifecycle and the most important invalid-input scenarios. A successful test run should preserve valid data in MySQL after the Java program terminates and restarts.

Implementation Quality Check

- SQL operations are isolated inside ContactDAO.
- User-driven SQL values are bound through PreparedStatement.
- JDBC resources are managed with try-with-resources.
- The project contains only the classes required for the stated scope.
- No Spring, Hibernate, REST API, React, or other unnecessary framework has been introduced.
- The code remains compact enough for a student to understand and explain during viva.

CONCLUSION

The Contact Management System successfully demonstrates how Core Java can be combined with JDBC and MySQL to build a practical database-driven application. The system supports the complete contact lifecycle: adding records, viewing them, searching by name or phone, updating records, and deleting records.

The most important design decision was to keep the implementation deliberately small. Instead of creating a large framework-based application, the project uses four primary Java classes and a single relational table. This makes the code easier to understand, test, debug, and explain while still demonstrating OOP, JDBC, SQL, validation, and exception handling.

PreparedStatement provides parameterized SQL execution, while try-with-resources helps ensure JDBC resources are closed correctly. The DAO layer keeps SQL away from the main console workflow. These practices give the project a cleaner structure without turning a simple college application into an unnecessarily complex enterprise system.

The current implementation is intentionally limited to a local console environment. Future versions can introduce a graphical or web interface, authentication, REST APIs, ORM, and import/export facilities. Such extensions should be added only when the project requirements justify them.

REFERENCES

1. **E. Balagurusamy**, *Programming with Java: A Primer*, 6th ed., McGraw Hill Education, 2019.
2. **Herbert Schildt**, *Java: The Complete Reference*, 12th ed., Oracle Press / McGraw-Hill, 2021.
3. **Cay S. Horstmann**, *Core Java Volume I – Fundamentals*, 12th ed., Prentice Hall, 2022.
4. **Oracle Corporation**, *Java Platform, Standard Edition 17 API Specification - java.sql Package*, Oracle Documentation.
5. **Oracle Corporation**, *MySQL Connector/J 8.4 Developer Guide*.
6. **Robert C. Martin**, *Clean Code: A Handbook of Agile Software Craftsmanship*, Prentice Hall, 2008.
7. **University of Mumbai**, *Curriculum for Second Year Engineering (AIML) – Full Stack Java Programming (FSJP - 2113611)*, Academic Year 2026–2027.

Report preparation note

The first four pages of this final document are preserved from the supplied TSEC FSJP report template. The technical report content after those pages has been revised to match the Contact Management System project specification and the supplied project-report source document. The report uses execution traces instead of claiming that generated console text is a real screenshot.